Laboratório Funcional · Aula 12

Lucas Gabriel Godoy de Oliveira · RA 24214463-2 · Turma ESOFT6S-C
Gerado em 16/10/2026, 20:33:21 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1 - agora() é impura, pois utiliza datetime.now() para obter a data e a hora atuais. O resultado depende do momento em que a função é chamada, podendo ser diferente a cada execução.
2 - registra(msg) é impura, pois utiliza print(msg) para escrever uma mensagem na tela, causando um efeito colateral. A função não apenas retorna um valor, mas também altera o que é exibido ao usuário.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

4: 14

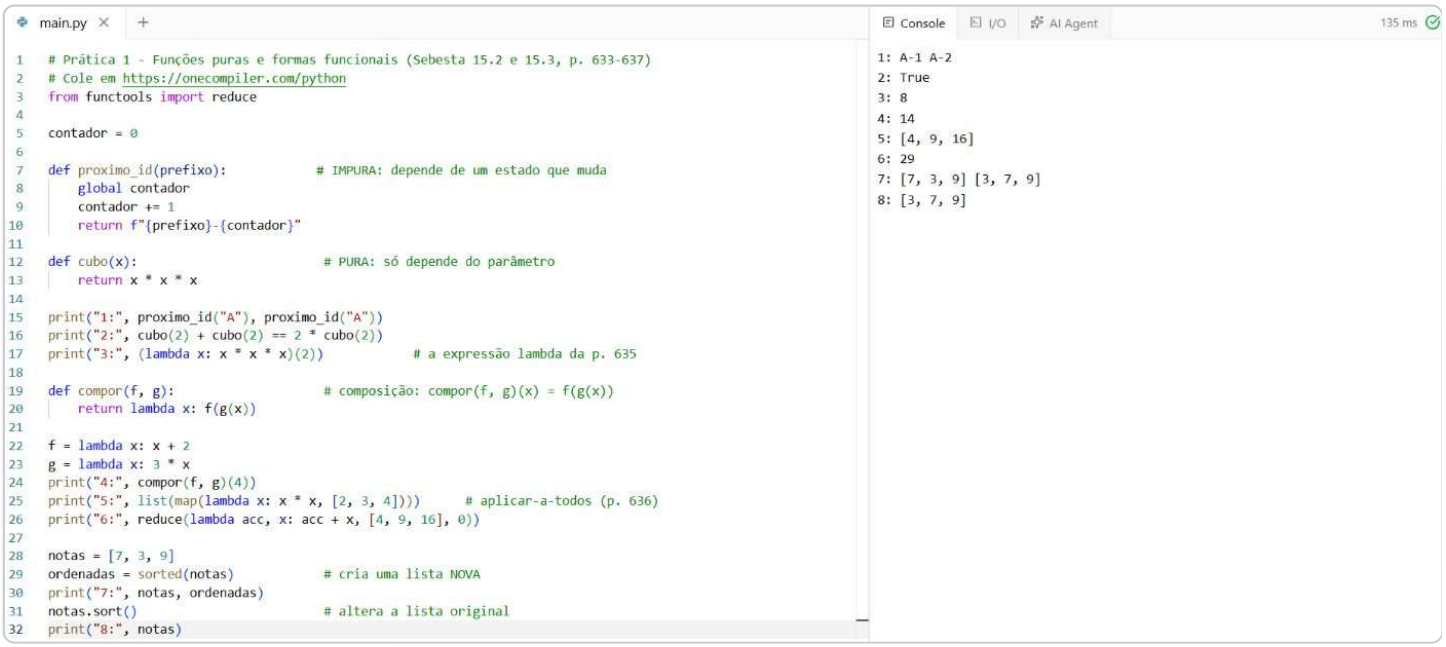
7: [7, 3, 9] [3, 7, 9]

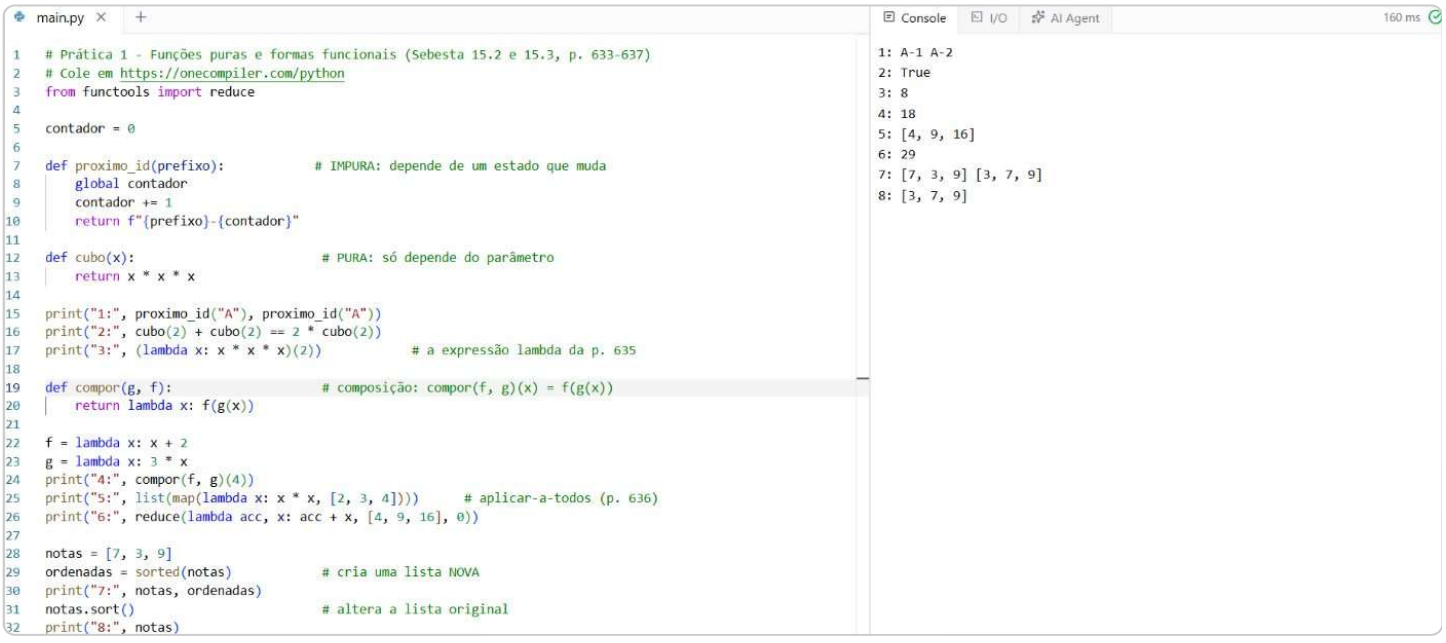
Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]


1: A-1 A-2
```

```
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes porque depende da variável global contador, que aumenta a cada chamada. Isso dificulta os testes porque o resultado depende do estado anterior do programa, tornando os testes menos previsíveis e reproduzíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4): primeiro, g(4) = 3 × 4 = 12. Depois, f(12) = 12 + 2 = 14.
compor(g, f)(4): primeiro, f(4) = 4 + 2 = 6. Depois, g(6) = 3 × 6 = 18.

Os resultados são diferentes (14 e 18) porque a ordem de aplicação das funções muda. A composição de funções não é comutativa.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e devolve uma nova lista ordenada, sem modificar a lista original. Já notas.sort() ordena a própria lista, modificando-a. sorted(notas) segue melhor o estilo funcional porque preserva a lista original e evita modificar um estado existente. Já notas.sort() produz um efeito colateral ao alterar a lista.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

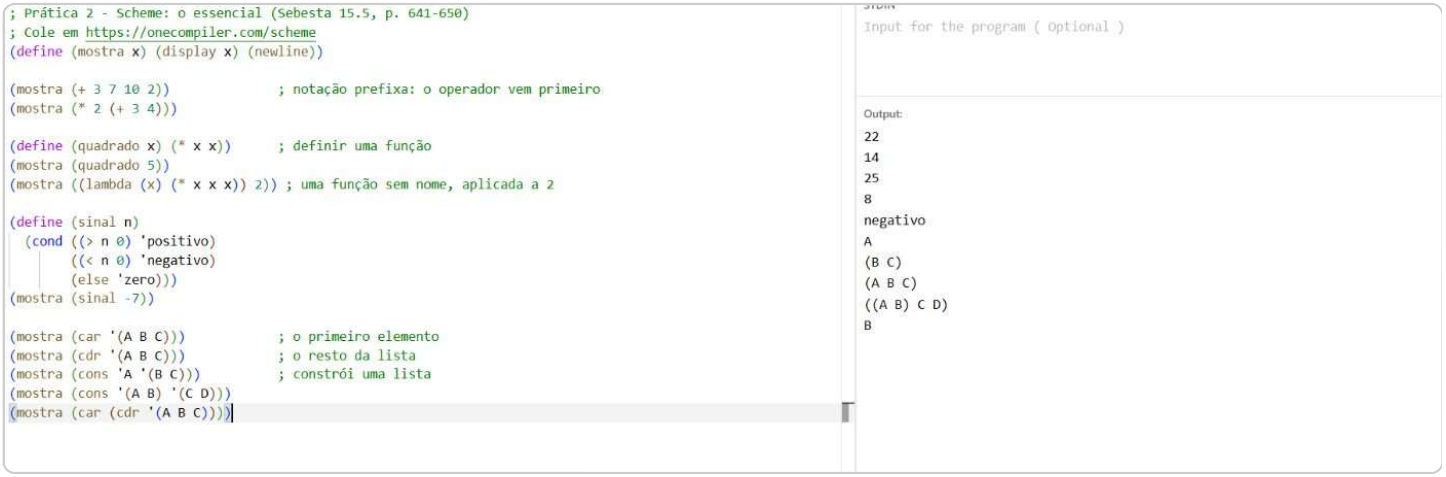
((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B


22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) b4b7aa72c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) b4b…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```



```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

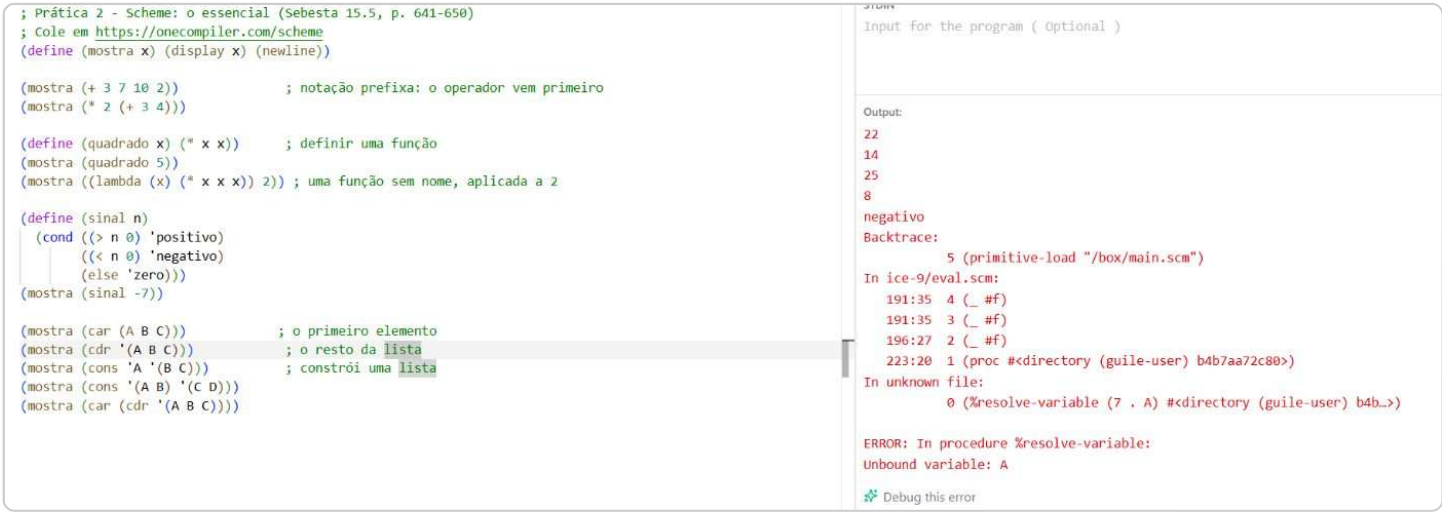
(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car '(A B C)))            ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

```
Input for the program ( Optional )

Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```



```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car (A B C)))             ; o primeiro elemento
(mostra (cdr '(A B C)))             ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

```
Input for the program ( Optional )

Output:
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) b4b7aa72c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) b4b...>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
Debug this error
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') serve para indicar que o Scheme deve tratar uma expressão como dado, sem tentar executá-la. Sem ele, o Scheme tentou interpretar (A B C) como uma chamada de função, considerando A como a função e B e C como argumentos. Como A não está definida como função, ocorreu um erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
Explicação: O primeiro cdr remove A, deixando (B C). O segundo cdr remove B, deixando (C).
Por fim, car devolve o primeiro elemento, C.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme: provavelmente vai funcionar, pois conta-ate usa recursão em cauda. O sistema pode otimizar as chamadas e reutilizar o espaço na pilha, evitando acumular um milhão de registros de ativação. Em Python: provavelmente ocorrerá um RecursionError, porque Python tem um limite de profundidade de recursão e não otimiza chamadas em cauda automaticamente.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```



Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```



```
# Prática 3b - A mesma recursão em cauda, agora em Python
# Cole em https://onecompiler.com/python
def conta_ate(n, acc):
    if n == 0:
        return acc
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação

print(conta_ate(500, 0))
print(conta_ate(1_000_000, 0))
```

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded

Debug this error
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a pilha de execução, pois cada chamada precisa esperar a chamada seguinte terminar para realizar a operação pendente, como a multiplicação do fatorial. Na recursão em cauda, não há operações pendentes após a chamada recursiva, permitindo que o Scheme reutilize o registro de ativação e mantenha o uso de memória constante.

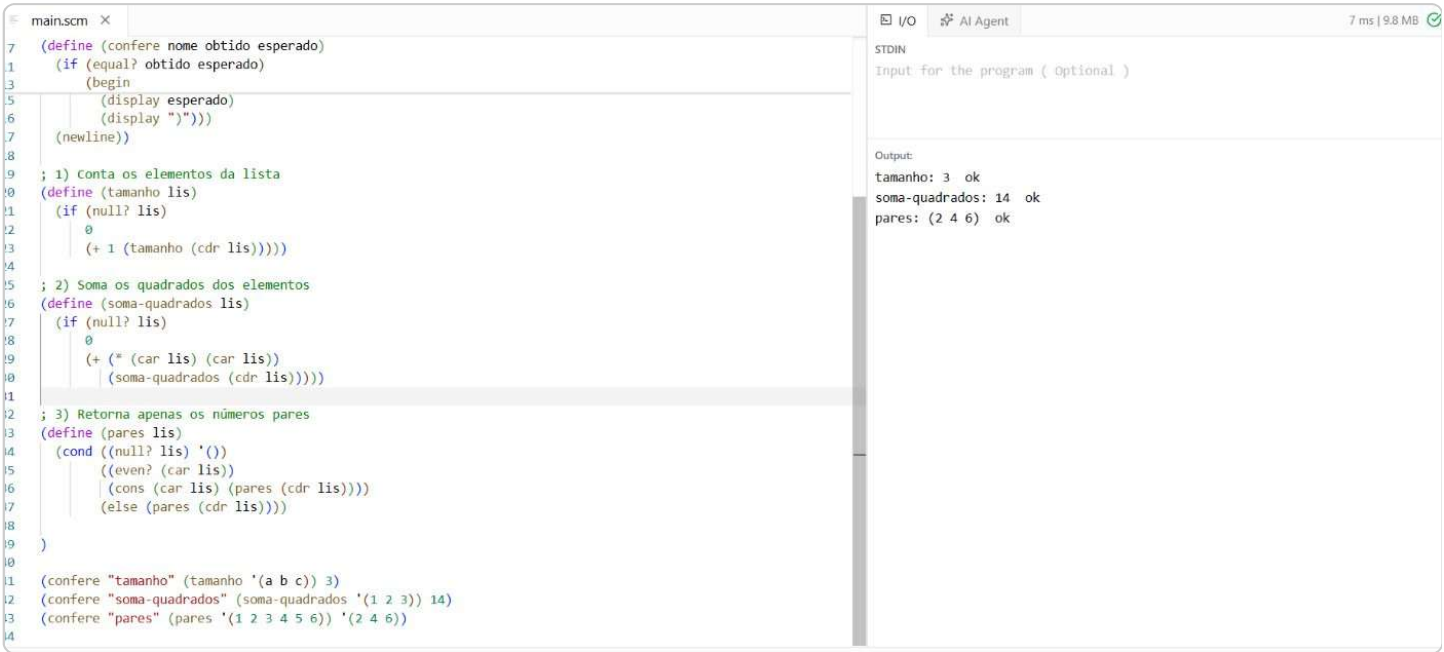
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu realizar um milhão de chamadas porque otimiza a recursão em cauda, reutilizando o espaço da pilha. Já o Python não faz essa otimização automaticamente e possui um limite de profundidade de recursão. Por isso, ao atingir esse limite, ele gera o erro RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```



```
(define (confere nome obtido esperado)
  (if (equal? obtido esperado)
      (begin
        (display esperado)
        (display ")")))
  (newline))

; 1) Conta os elementos da lista
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

; 2) Soma os quadrados dos elementos
(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

; 3) Retorna apenas os números pares
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))

)

(confere "tamanho" (tamanho '(a b c)) 3)
(confere "soma-quadrados" (soma-quadrados '(1 2 3)) 14)
(confere "pares" (pares '(1 2 3 4 5 6)) '(2 4 6))
```

```
I/O  AI Agent                                   7 ms | 9.8 MB
STDIN
Input for the program ( Optional )

Output:
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como usar a recursão para percorrer a lista até ela ficar vazia. Em todas as funções, null? verifica se a lista acabou, car acessa o primeiro elemento e cdr retorna o restante da lista.
tamanho: retorna 0 se a lista estiver vazia; caso contrário, soma 1 ao tamanho do restante.
soma-quadrados: retorna 0 se a lista estiver vazia; caso contrário, calcula o quadrado do primeiro elemento e soma ao resultado do restante.
pares: retorna uma lista vazia quando não há mais elementos. Se o primeiro elemento for par, cons o adiciona ao resultado; caso contrário, ele é ignorado.

Prática 5: Haskell: padrões, compreensões e preguiça

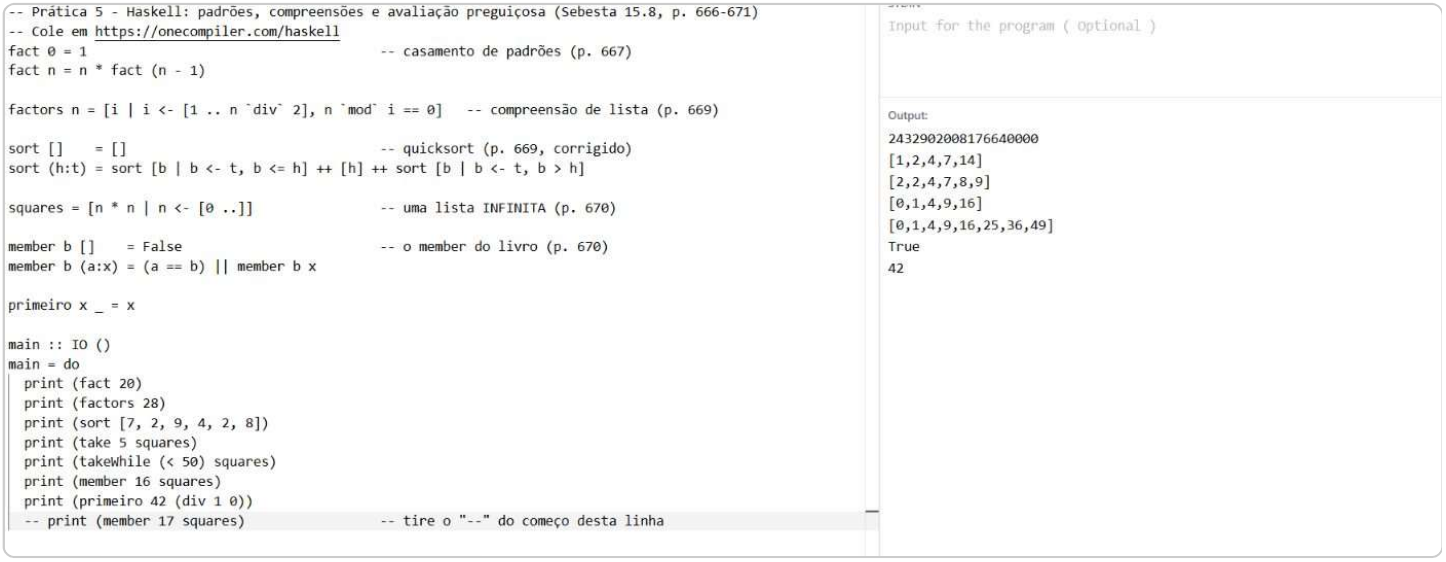
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16], pois pega os cinco primeiros quadrados da lista infinita: 0^2, 1^2, 2^2, 3^2, 4^2.
primeiro 42 (div 1 0) imprime 42, porque o Haskell usa avaliação preguiçosa. Como a função primeiro ignora o segundo parâmetro (_), a divisão por zero nunca é calculada e, portanto, não ocorre erro.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42


Time limit exceeded
```



```haskell
-- Prática 5 - Haskell: padrões, compreensões e avaliação preguiçosa (Sebesta 15.8, p. 666-671)
-- Cole em https://onecompiler.com/haskell
fact 0 = 1                                  -- casamento de padrões (p. 667)
fact n = n * fact (n - 1)

factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]   -- compreensão de lista (p. 669)

sort []    = []                             -- quicksort (p. 669, corrigido)
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]

squares = [n * n | n <- [0 ..]]             -- uma lista INFINITA (p. 670)

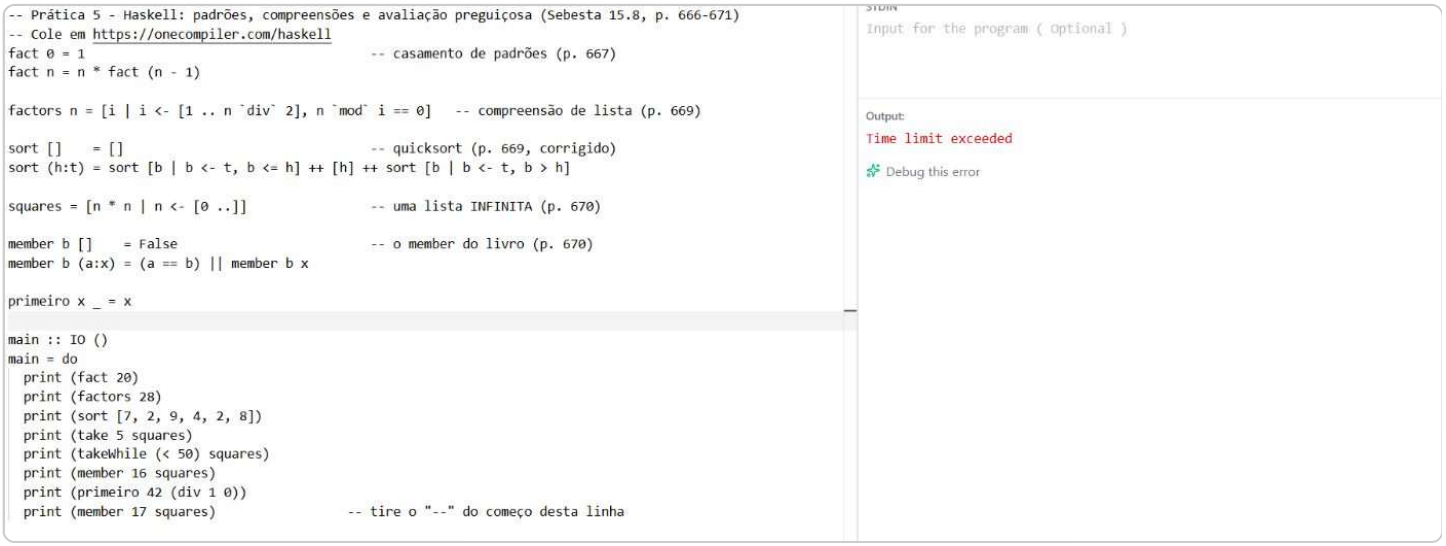
member b []    = False                      -- o member do livro (p. 670)
member b (a:x) = (a == b) || member b x

primeiro x _ = x

main :: IO ()
main = do
  print (fact 20)
  print (factors 28)
  print (sort [7, 2, 9, 4, 2, 8])
  print (take 5 squares)
  print (takeWhile (< 50) squares)
  print (member 16 squares)
  print (primeiro 42 (div 1 0))
  -- print (member 17 squares)              -- tire o "--" do começo desta linha
```

```
Input for the program ( Optional )

Output:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```



```haskell
-- Prática 5 - Haskell: padrões, compreensões e avaliação preguiçosa (Sebesta 15.8, p. 666-671)
-- Cole em https://onecompiler.com/haskell
fact 0 = 1                                  -- casamento de padrões (p. 667)
fact n = n * fact (n - 1)

factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]   -- compreensão de lista (p. 669)

sort []    = []                             -- quicksort (p. 669, corrigido)
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]

squares = [n * n | n <- [0 ..]]             -- uma lista INFINITA (p. 670)

member b []    = False                      -- o member do livro (p. 670)
member b (a:x) = (a == b) || member b x

primeiro x _ = x

main :: IO ()
main = do
  print (fact 20)
  print (factors 28)
  print (sort [7, 2, 9, 4, 2, 8])
  print (take 5 squares)
  print (takeWhile (< 50) squares)
  print (member 16 squares)
  print (primeiro 42 (div 1 0))
  print (member 17 squares)              -- tire o "--" do começo desta linha
```

```
Input for the program ( Optional )

Output:
Time limit exceeded

Debug this error
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita por causa da avaliação preguiçosa. Ele calcula apenas os elementos necessários. Assim, take 5 squares calcula somente os cinco primeiros quadrados. A expressão primeiro 42 (div 1 0) não deu erro porque a função primeiro ignora o segundo parâmetro (_). Como esse valor não é necessário, a divisão por zero nunca é calculada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares termina porque o número 16 aparece na lista de quadrados. Já member 17 squares nunca termina porque 17 não é um quadrado perfeito e a função continua procurando em uma lista infinita.
Como corrigir: como squares está em ordem crescente, podemos interromper a busca quando o elemento atual ultrapassar o valor procurado.

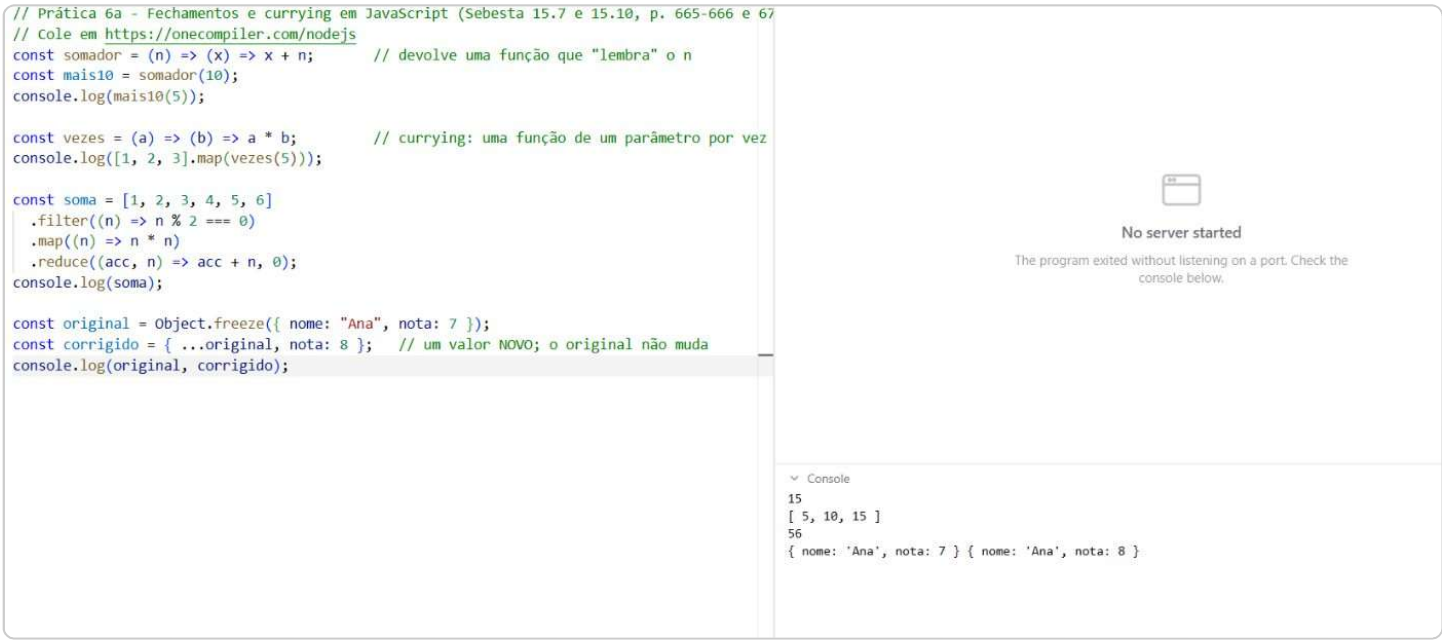
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15.
[1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```



Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida ainda lembra o valor 10 porque é um fechamento (closure). Ela mantém uma referência ao ambiente em que foi criada, preservando o valor de n, mesmo depois que somador(10) terminou de executar. Assim, mais10(5) calcula 5 + 10 = 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque a ordem de aplicação das funções muda:

f.compose(g).apply(4) calcula primeiro g(4) = 12 e depois f(12) = 14.

f.andThen(g).apply(4) calcula primeiro f(4) = 6 e depois g(6) = 18.

O método compose corresponde a compor(f, g) da Prática 1, pois aplica primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244

ANA, CARLA, DAVI
```

```java
// Prática 7 - Exercício 1: reescreva o laço como um pipeline (stream)
// Cole em https://onecompiler.com/java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = 0;
        for (int n : nums) {
            if (n % 2 == 0) {
                soma += n * n;
            }
        }
        System.out.println(soma);
    }
}
```

```
244
```

```javascript
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) => aluno.nota >= 6)
  .map((aluno) => aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

```
Console
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler porque deixa claro o que queremos fazer com os dados, sem precisar controlar manualmente os laços e as variáveis acumuladoras. A versão com laço pode ser mais fácil de depurar em alguns casos, pois permite acompanhar cada iteração e verificar os valores intermediários passo a passo. Já na versão funcional, podemos testar separadamente as etapas de filtragem e transformação.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em C é mais fácil de entender, pois utiliza um laço for e uma variável acumuladora, tornando mais claro o passo a passo do cálculo. Eu aceitaria o custo de execução em sistemas simples, nos quais o desempenho não é tão importante, mas não em sistemas que exigem alta velocidade, como jogos em tempo real.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que é possível realizar as mesmas tarefas sem utilizar laços e variáveis acumuladoras, por meio da programação funcional.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

não respondido